Nearest neighbor searching on the GPU

B.Tech Project
Stage 2

by

Aditya Bhosale
140010009

under the guidance of

Prof. Prabhu Ramachandran

Department of Aerospace Engineering
Indian Institute of Technology, Bombay
Mumbai 400 076

Abstract

Nearest neighbor searching is one of the bottlenecks in most particle based simulations. Nearest neighbor searching takes about 40% of the time in an SPH simulation. In this report we look at implementation of nearest neighbor searching algorithms on the GPU.

General-purpose computing on graphics processing units (GPGPU) is the use of a graphics processing unit (GPU), which typically handles computation only for computer graphics, to perform computation in applications traditionally handled by the central processing unit (CPU) [Wikipedia, 2017]. GPUs generally work at lower frequencies than CPUs but can spawn a large number of threads. Thus, array based operations can be efficiently parallelized on a GPU.

One of the most widely used data structures for nearest neighbor searching for particle-based simulations is the uniform grid. This report looks at OpenCL implementation of space-filling curves based uniform grids for nearest neighbor searching. We highlight the implementational challenges for running the algorithm on the GPU and propose solutions to them.

We also look at an OpenCL implementation of stratified grids which can be used to find neighbors in case of variable search radius efficiently.

Our OpenCL implementation is then compared to the previous serial Cython implementation using a standard benchmark.

Contents

1 Introduction **1**

1.1 Parallel computation on GPU 1

1.2 Overview 2

2 Background **3**

2.1 Spatial Hash 3

2.2 Linked List 3

2.3 Cell Indexing 3

2.4 Space Filling Curves 4

2.5 Extended uniform grids 5

2.6 Stratified Grids 5

2.7 Octree 5

3 Framework **7**

3.1 Basic idea 7

3.2 Parallelism 7

3.3 Interfacing with the CPU 7

3.4 Code generation using `mako` and `pyopencl` 9

4 Space-filling curves **10**

4.1 Building the uniform grid 10

4.2 Reducing memory requirement 10

4.3 Optimized binary search 11

4.4 Handling multiple particle arrays 12

5 Stratified Grids **13**

5.1 Building the stratified grid 13

5.2 Disadvantages of using stratified grids 13

6 Results **15**

6.1 Comparison of build times 15

6.1.1 Comparison of SFC on GPU with Cython implementation 15

6.1.2 Comparison of SFC OpenCL implementation on CPU with Cython implementation 15

6.2 Comparison of nearest neighbor search times 15
6.2.1 Constant search radius 15
6.2.2 Variable search radius 19

7 Summary **22**

List of Figures

1.1 Nearest neighbors of a particle with search radius r 1

2.1 Space Filling curve for a 3D grid [Wikimedia, 2008] 4
2.2 Comparison of cell indexing and space-filling curves 5
2.3 Stratified grids with 4 levels 6
2.4 Example of an Octree 6

3.1 Overview of framework used for implementation 8

4.1 Comparison of SFC with binary search method and optimized binary search method 11

5.1 Different keys are generated for the same cell at different levels. The sorted array has separated sub-arrays for each level. 14

6.1 Comparison of build times for SFC on GPU and CPU using Cython 16
6.2 Comparison of SFC OpenCL implementation on CPU with Cython implementation 17
6.3 Comparison of SFC on GPU with Cython implementation. Shows (a) speed up (b) time taken 18
6.4 Comparison of SFC OpenCL implementation on CPU with Cython implementation. Shows (a) speed up (b) time taken 18
6.5 Dataset with a gradient of search radius from $h_{\min}$ to $h_{\max}$ 19
6.6 Comparison of SFC on GPU with Octree in serial on CPU for $\frac{h_{\max}}{h_{\min}} = 5$ 20
6.7 Comparison of SFC with Stratified Grids on the GPU for $\frac{h_{\max}}{h_{\min}} = 5$ and 4 levels in the stratified grids. 21

Chapter 1

Introduction

Nearest neighbor searching is one of the bottlenecks in most particle based simulations. For instance, nearest neighbor searching takes about 40% of the time in an SPH simulation. In this report we look at implementation of nearest neighbor searching algorithms on the GPU.

The neighbor set of a particle q is defined as follows,

$$\mathcal{N}(q) = \{\ j\ \forall\ j \in \mathbb{S}\ |\ ||\mathbf{r}_q - \mathbf{r}_j|| < \max(h_q, h_j)\}$$

1.1 Parallel computation on GPU

General-purpose computing on graphics processing units (GPGPU) is the use of a graphics processing unit (GPU), which typically handles computation only for computer graphics, to perform computation in applications traditionally handled by the central processing unit (CPU) [Wikipedia, 2017].

GPUs generally work at lower frequencies than CPUs but can spawn a large number of threads. Thus, array based operations can be efficiently parallelized on a GPU.

GPGPU can be used for speeding up SPH simulations significantly. In this report we will study the performance improvement acheived using GPU implementation for nearest neighbor searching which is the biggest bottleneck in an SPH simulation.

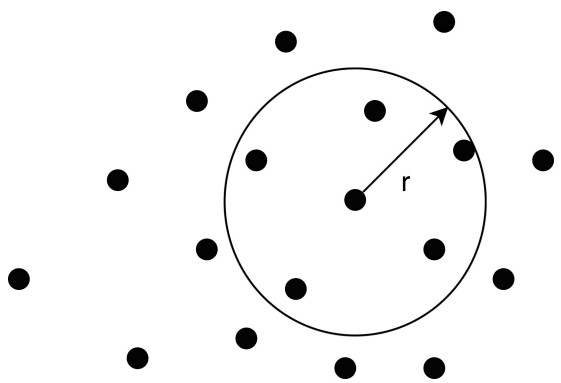


Figure 1.1: Nearest neighbors of a particle with search radius r

Open Computing Language (OpenCL) is a framework for writing programs that execute across heterogeneous platforms. Thus, the same code can run on CPUs as well as GPUs. We use `pyopencl` [Klöckner et al., 2012] for the GPU implementation.

1.2 Overview

Chapter 2 gives an overview of the previous work done in nearest neighbor searching.

In chapter 3, we discuss the challenges in implementing these algorithms on the GPU and present the framework used for the implementation.

In chapter 4, we present an algorithm to build space-filling curves on the GPU. We also present alternative methods to reduce the memory requirement and get resonable performance.

In chapter 5, we present an algorithm to build stratified grids on the GPU. We also discuss the disadvantages of using stratified grids.

In chapter 6 we compare the OpenCL implementation run on a GPU as well as CPU with our previous Cython implementation run in serial.

Chapter 2

Background

Nearest neighbor searching for particle based simulations typically use uniform grids or heirarchical algorithms. This section will give a brief review of the work done during BTP-1 on algorithms using both uniform grids and the octree.

2.1 Spatial Hash

In this method each cell in the uniform grid is mapped to an index in a hashtable. Following hash function is widely used for this purpose [Onderik and Durikovic, 2008],

$$\hat{H}(i, j, k) = (i.p_1 \oplus j.p_2 \oplus k.p_3) \bmod M$$

where $\hat{H}(i, j, k)$ is the hash value for cell (i, j, k), M is the table size of the hashtable and $p_1 = 73856093$, $p_2 = 19349663$ and $p_3 = 83492791$.

Spatial hash gives a poor cache performance as the hashtable is randomly spread across the memory.

2.2 Linked List

In this method [Dominguez et al., 2011], the cell id's are flattened to map a cell to an index in an array which stores the first particle in that cell. Another array then maps the particle to the next particle in that cell.

Linked list algorithm gives a better cache performance because of it's compactness. However, empty cells are also stored in this algorithm thus increasing it's memory requirements.

2.3 Cell Indexing

This algorithm uses a sorted array to access particles in a cell. The particles are sorted according to the keys given by the following function [Onderik and Durikovic, 2008],

$$\hat{H}(n, i, j, k) = n + 2^I i + 2^{I+J} j + 2^{I+J+K} k$$

such that,

$$
\begin{aligned}
2^I &> N \\
2^J &> \left\lfloor \frac{B_x}{h} \right\rfloor \\
2^K &> \left\lfloor \frac{B_y}{h} \right\rfloor \\
2^L &> \left\lfloor \frac{B_z}{h} \right\rfloor
\end{aligned}
$$

Observe that, $\hat{H}(n, i_n, j_n, k_n) \leq \hat{H}(m, i_m, j_m, k_m) \implies k_n \leq k_m$. Also when $k_n = k_m$, $j_n \leq j_m$ and so on.

The particles can then be accessed by storing the indices in the sorted array where the first particle of a cell (ie. the particle in the cell with least particle id) is stored.

Cell indexing thus gives a better cache performance without the large memory requirement of a linked list.

2.4 Space Filling Curves

Space Filling Curves or Z-Index sort [Goswami et al., 2010], similar to Cell Indexing, can be used to map 3D cells to 1D array by sorting the list of particles according to it's hash values using bit interleaving.

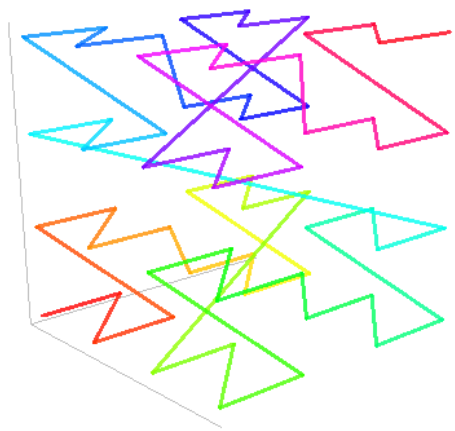


Figure 2.1: Space Filling curve for a 3D grid [Wikimedia, 2008]

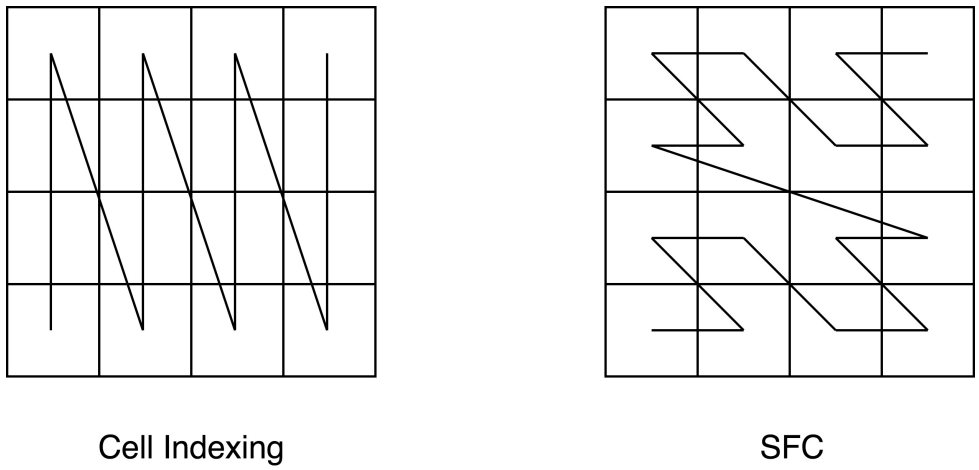


Figure 2.2: Comparison of cell indexing and space-filling curves

From figure 2.2 note that space filling curves give a better cache performance than cell indexing as the neighboring cells are closer in memory in case of SFC than in case of cell indexing.

2.5 Extended uniform grids

Extended uniform grids work by sub-dividing the grid and searching for neighbors on a sub-cell resolution. This method was found to be useful in cases of variable search radius with a large number of particles.

2.6 Stratified Grids

Stratified grids solves the problem of large bin sizes by using multiple grids of different cell sizes. In stratified grids, the particles are classified into different levels based on their search radius and then into bins based on spatial location. The neighbors are then found by querying for neighbors on each level. Figure 2.3 shows a stratified grid with 4 levels.

2.7 Octree

Octree is a 3D analog to 1D binary tree and 2D quadtree. Each node in an octree has a maximum of eight children.

A tree based implementation [Hernquist and Katz, 1989] can be used to efficiently find neighbors in cases of variable support radius.

Hashtables and linked lists are inherently serial data structures and while it could be possible to parallelize them to some extent, they will always have a serial component to them. The cell indexing and space-filling curve however can be efficiently parallelized as we will see in the following chapters.

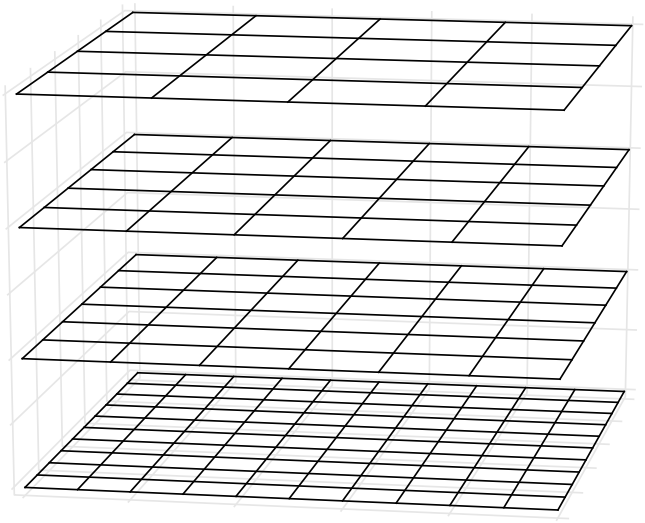


Figure 2.3: Stratified grids with 4 levels

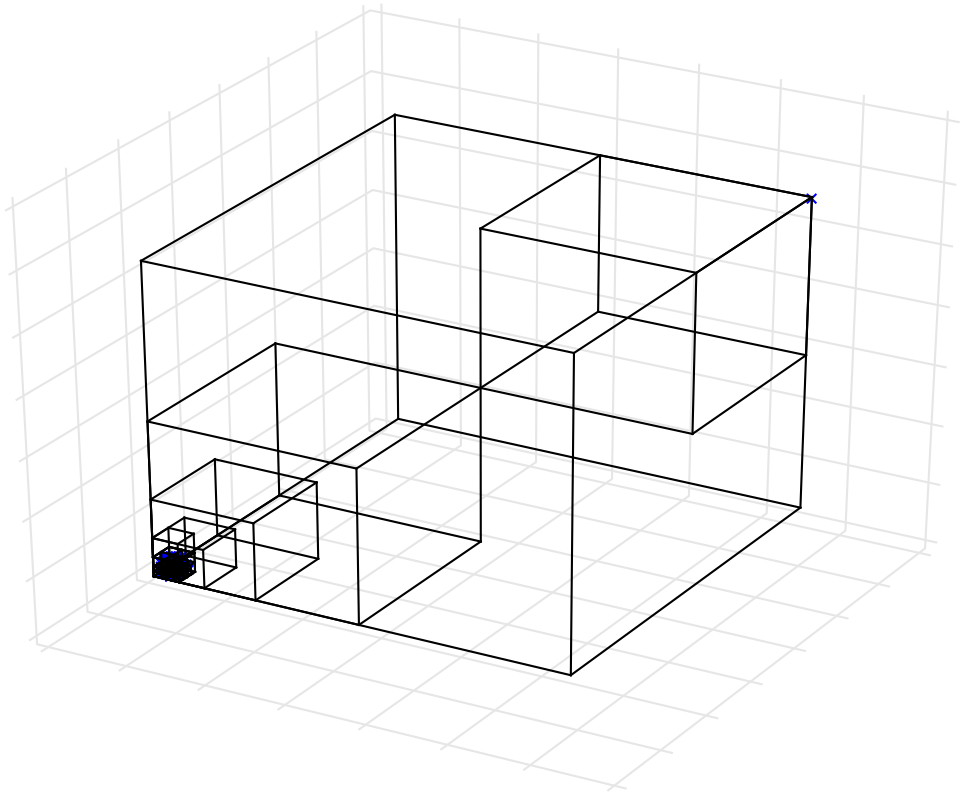


Figure 2.4: Example of an Octree

Chapter 3

Framework

Using GPUs for computation immediately puts a number of restrictions on what operations can be performed while querying for neighbors. For instance, dynamic memory allocation, which is an essential operation while querying for neighbors, is not allowed on GPUs. Thus an alternate approach needs to be adopted.

The framework used for implementing nearest neighbor searching algorithms on the GPU will be discussed in this chapter.

3.1 Basic idea

As dynamic memory allocation is not allowed, neighbors need to queried for twice; the first time for finding the number of neighbors per particle and the second time for actually finding the neighbors. After the first query, the memory required for storing all the neighbors is allocated, thus getting rid of the dynamic memory allocation aspect of the query. An overview of how this is done is shown in figure 3.1.

3.2 Parallelism

Each implementation is a derived class of `GPUNNPS` and is required to overload the two methods, `find_neighbor_lengths` and `find_nearest_neighbors_gpu`.

`find_neighbor_lengths` calls an OpenCL kernel with number of work items equal to the number of particles. A work item i finds the number of neighbors for particle i in the destination array. `find_nearest_neighbors_gpu` follows the same approach but stores the neighbors instead of just counting the number.

3.3 Interfacing with the CPU

The neighbors array on the GPU can be copied to the CPU as a numpy array. The original `pysph.base.nnps` [Ramachandran and Puri, 2013] API uses an output parameter of type `UIntArray` for finding neighbors. To keep the same API, the output parameter's view is set using the numpy array data pointers.

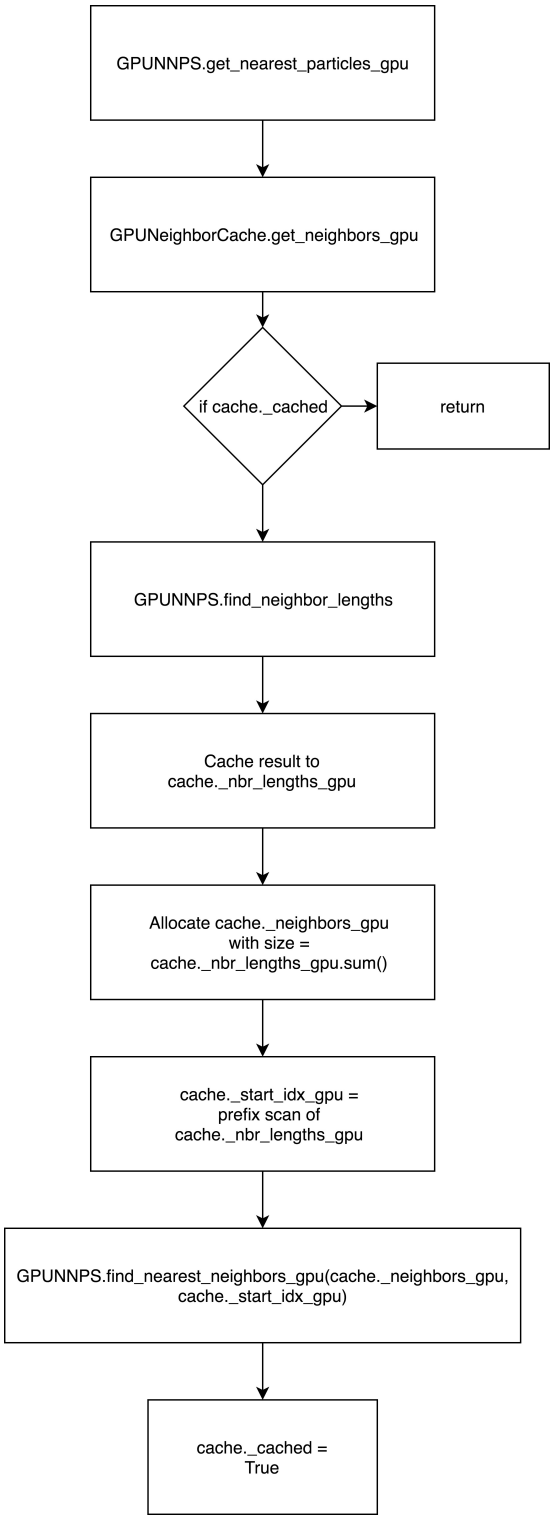


Figure 3.1: Overview of framework used for implementation

3.4 Code generation using `mako` and `pyopencl`

Some of the GPUs in the market do not support double precision. Our implementation handles this problem by having a parameter called `use_double` passed to the `GPUNNPS` class. We then generate OpenCL code based on that argument. The python-based template library `mako` is used for generating this code.

We use `pyopencl`'s `ElementwiseKernel` to generate the final OpenCL kernel. The `ElementwiseKernel` handles the job of assigning the number of work items and work groups.

When the algorithm is used for the first time, the OpenCL kernel code is generated and compiled at runtime and `pyopencl` then caches the compiled binaries. The next time it is used, the compiled code will be used directly.

Chapter 4

Space-filling curves

Space-filling curve uses a sorted array for storing the uniform grid. Each cell is assigned a unique key and the particle indices are then sorted according to the key assigned to the cell they are in.

4.1 Building the uniform grid

The backbone of SFC based nearest neighbor searching is the idea that each cell can be mapped to a unique key which can be then used to sort the particles in a particular order (see figure 2.1).

We allocate two arrays for storing particle index and key corresponding to that particle. A kernel then fills both the arrays with a work item i handling particle i. Both the arrays are then sorted according to the keys array. `pyopencl`'s implementation of radix sort is used for sorting.

For querying for neighbors, particles from 27 neighboring boxes need to be checked. The keys corresponding to the neighboring cells can be easily found. Thus, we need a way to map the key of each cell to the first particle in the sorted array of particle indices belonging to that cell.

One way of doing this is to allocate an array of size equal to one more than the largest key in the keys array. This array can then be filled using a kernel which checks if the key corresponding to the work item i is the same as the key corresponding to work item $i - 1$.

We can see that this has a large memory requirement as empty cells will also be stored causing the array size to blow up for sparse datasets especially since this array will be stored on the GPU.

4.2 Reducing memory requirement

To solve the problem of large memory requirement, we make use of the fact that the arrays are sorted. Thus, we can use binary search to find the index corresponding to a key.

Using binary search is however a trade-off in speed for smaller memory requirement. There is a large performance overhead as each work item needs to perform 27 binary searches.

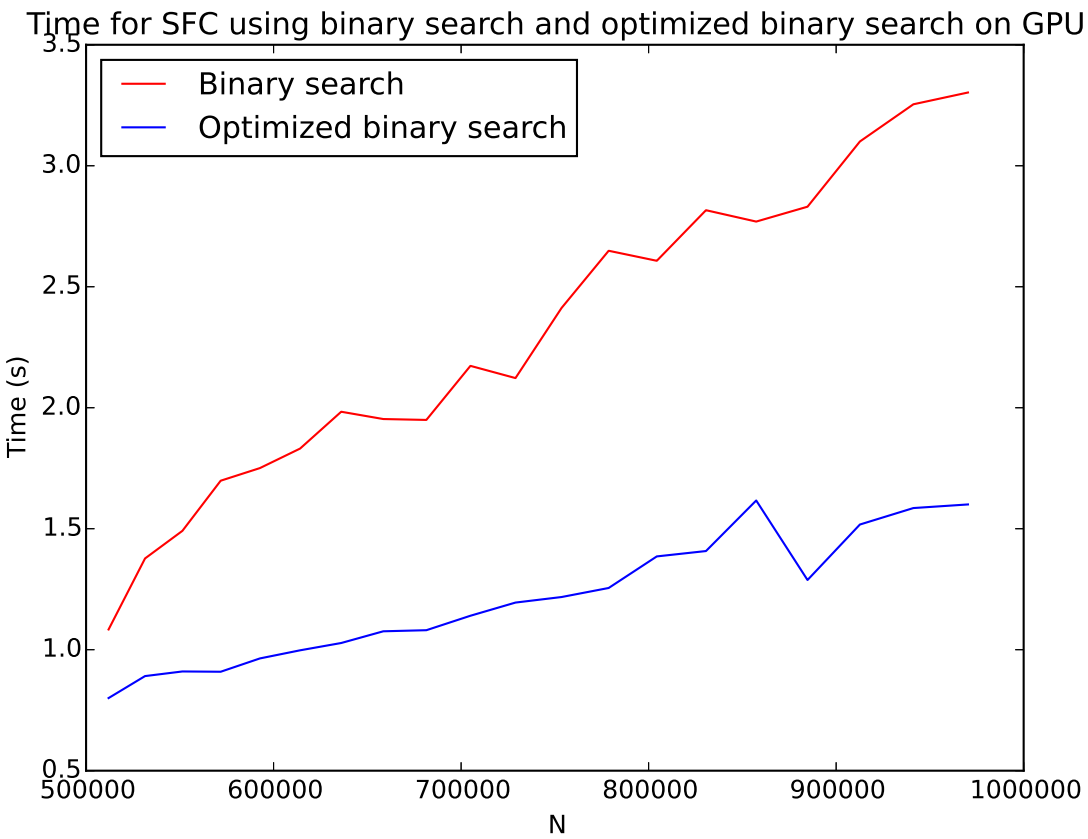


Figure 4.1: Comparison of SFC with binary search method and optimized binary search method

4.3 Optimized binary search

Recall that the large memory requirement was a result of storing empty cells. Thus if we map a cell to the index efficiently, we could get both, performance and smaller memory requirement.

One way would be to count the number of non-empty cells and assign a unique cell ID to each non-empty cell such that the unique id's are continuous integers. Then map the unique cell ID's to the start index of the cell it represents.

We allocate an array of size N for storing the unique cell ID's of each particle. A kernel then uses atomic operations to assign unique id's to each particle corresponding to it's key. Another array of size equal to $27M$, where M is the number of non-empty cells is then allocated to store the start indices of the 27 neighbors of each unique cell ID. Another kernel then performs 27 binary searches for each non-empty cell to find the start indices of it's neighboring cells.

The worst case space complexity for the above mentioned method is $\mathcal{O}(N)$. Thus, we can get rid of the large memory requirement. Also, note that the number of binary searches are reduced from $27N$ to $27M$ and as $M \leq N$, the overhead of binary searches is also reduced.

Figure 4.1 compares the performance of the binary search method with the optimized binary search method on the GPU. The optimized binary search shows a speed up of about 2.5x when finding neighbors.

4.4 Handling multiple particle arrays

A number of SPH problems need to use multiple particle arrays. For example, simulating a stone falling in water could use 2 particle arrays, one for the water (liquid) and one for the stone (solid). For calculating the force exerted by particles in one particle array on another, the neighbors of a query particle belonging to one array in another array need to be calculated.

We'll call the array to which the query particle belongs as the *destination* array and the array in which neighbors are sought as the *source* array. A combination of source and destination index is called a *context*.

Consider a set of particle arrays, $\{\mathbb{P}_1, \mathbb{P}_2, \ldots, \mathbb{P}_n\}$. The neighbors for source index i and destination index j would then be defined as,

$$\mathcal{N}_{ij}(q) = \{\ p \ \forall\ p \in \mathbb{P}_i \mid ||\mathbf{r}_q - \mathbf{r}_p|| < \max(h_q, h_p)\}$$

where $q \in \mathbb{P}_j$.

The building algorithm is run for each particle array and a separate uniform grid with the same cell size is made for each particle array. The query particle's cell is then located in the source uniform grid and the neighboring cells are found.

In the optimized binary search method, the unique cell ID assigned to each cell won't be the same for different particle arrays. Thus, another array is needed to map the cell ID in destination array to cell ID in source array. Also, because we are only storing neighboring cells of non-empty cells; which are calculated based on the source particle array only; the cell in the source grid to which the query particle belongs might be empty. Thus, another array for storing these overflow cells also need to be stored.

When the context is being set, ie. when the source and destination indices are set, we check for cells in the destination grid that are empty in the source grid. We then add those cells along with their 27 neighboring cells to the overflow cells array.

Chapter 5

Stratified Grids

For finding neighbors in cases of variable search radius, uniform grids can be slow because the space is binned at the maximum search radius. Stratified grids solves this problem by using multiple grids of different cell sizes. In stratified grids, the particles are classified into different levels based on their search radius and then into bins based on spatial location. The neighbors are then found by querying for neighbors on each level.

5.1 Building the stratified grid

Each level in the stratified grids is a separate space-filling curve. To differentiate the grids, a mask corresponding to the level is appended to the front of the key.

Thus, when the particle indices are sorted according to the keys, the grids are separated in the sorted arrays (see figure 5.1). Two more arrays are then used to store the start indices and number of particles in each level.

5.2 Disadvantages of using stratified grids

The neighborhood at level j is given by,

$$N(\mathcal{M}_j, q) = \{(s,t,u) \mid s,t,u \in \{-H(\mathcal{M}_j, q), \ldots, H(\mathcal{M}_j, q)\}\}$$

$$H(\mathcal{M}_j, q) = \left\lceil \frac{\max(h_q, f(\mathcal{M}_j))}{f(\mathcal{M}_j)} \right\rceil$$

where q is the query particle index, h_q is the search radius of the query particle, $\mathcal{M}_j$ is the grid at level j and $f(\mathcal{M}_j)$ is the cell size of the grid $\mathcal{M}_j$.

Thus, it can be seen that the number of neighboring cells to be checked for neighbors depends on the search radius of the query particle. Thus, the optimized binary search method won't work here as the size of the array to be allocated to map a cell to it's neighbors is not known.

We use the binary search method to find a cell in the sorted array which makes it significantly slower than the space-filling curve as it does more than $27n$ binary searches, n being the number of levels, per work item.

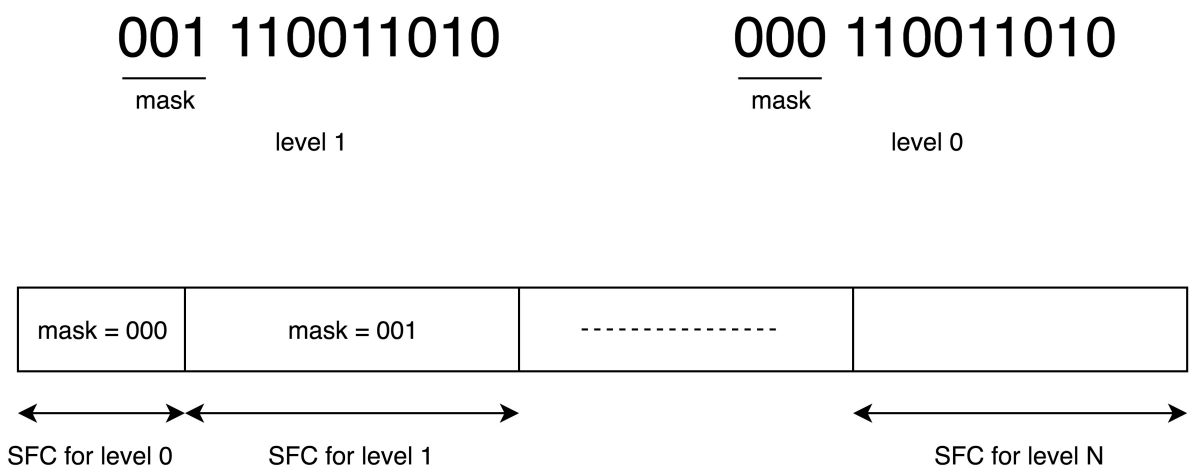


Figure 5.1: Different keys are generated for the same cell at different levels. The sorted array has separated sub-arrays for each level.

Chapter 6

Results

This chapter compares the build times and nearest neighbor search times for SFC and stratified grids.

All of the benchmarks have been run on an Intel Core i5 CPU (2 cores, 4 threads) and an Intel Iris 6200 GPU.

6.1 Comparison of build times

This section will compare the build times for SFC's OpenCL implementation on GPU as well as on CPU with the Cython implementation. The benchmarks have been run on a 3D uniform particle distribution.

6.1.1 Comparison of SFC on GPU with Cython implementation

Figure 6.1 shows a comparison of build times for SFC implementation on GPU and Cython implementation in serial. It can be seen that the GPU implementation is faster than the serial Cython CPU version.

6.1.2 Comparison of SFC OpenCL implementation on CPU with Cython implementation

Figure 6.2 shows a comparison of build times for SFC OpenCL implementation on CPU with Cython implementation in serial. As N increases, the parallel CPU version becomes faster than the serial Cython CPU implementation.

6.2 Comparison of nearest neighbor search times

6.2.1 Constant search radius

In this section, the algorithms will be with datasets of constant search radius such that $Nh^3 = k$. That is, the number of neighbors as N is scaled up stays the same.

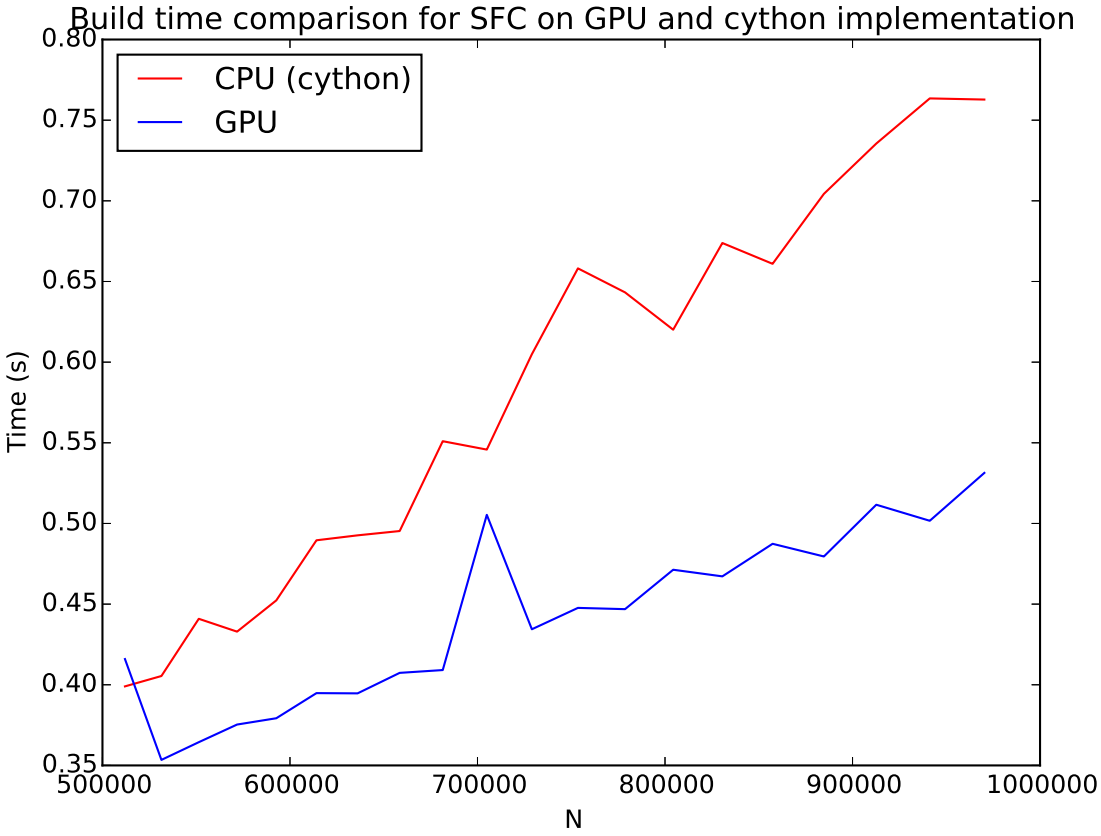


Figure 6.1: Comparison of build times for SFC on GPU and CPU using Cython

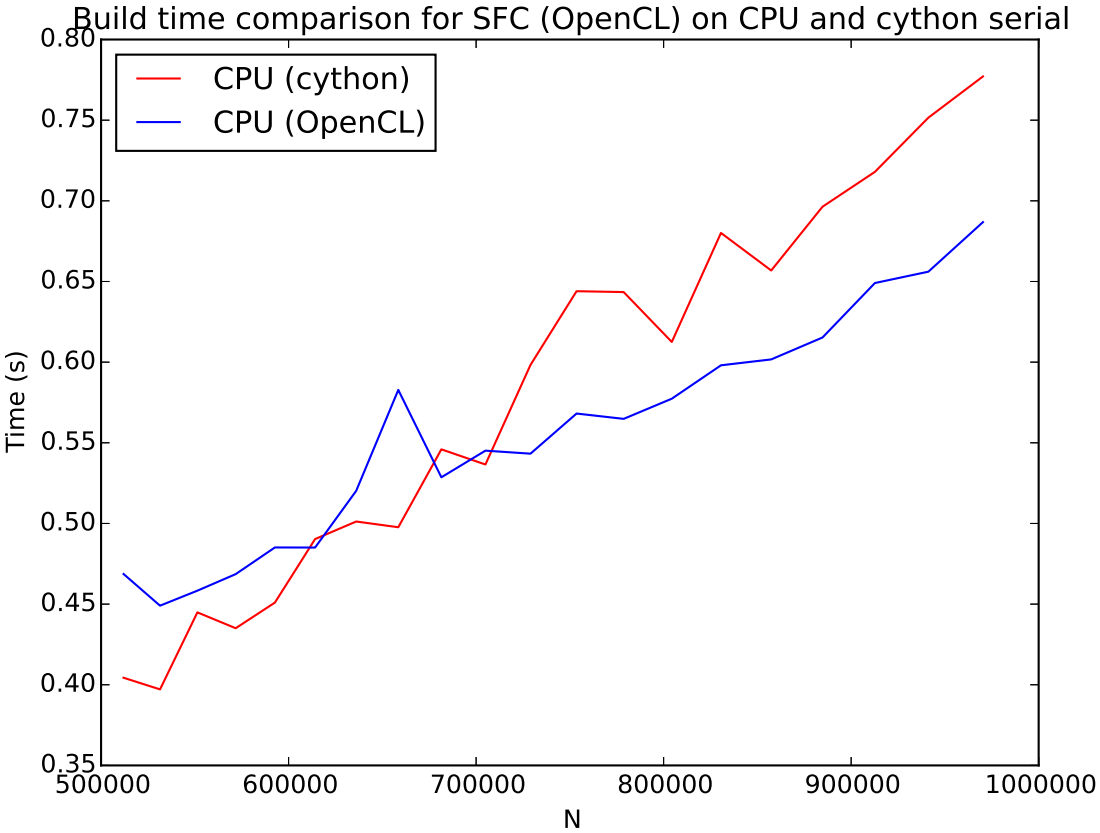
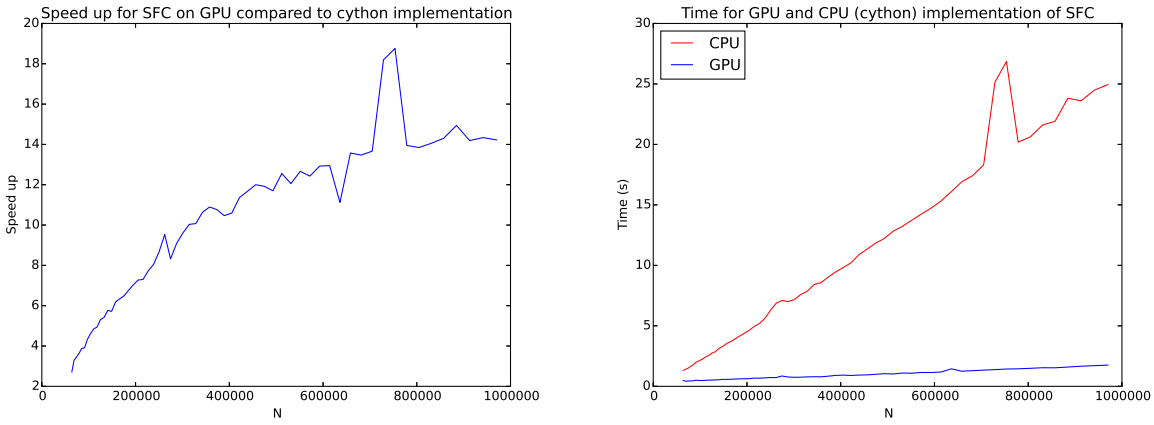
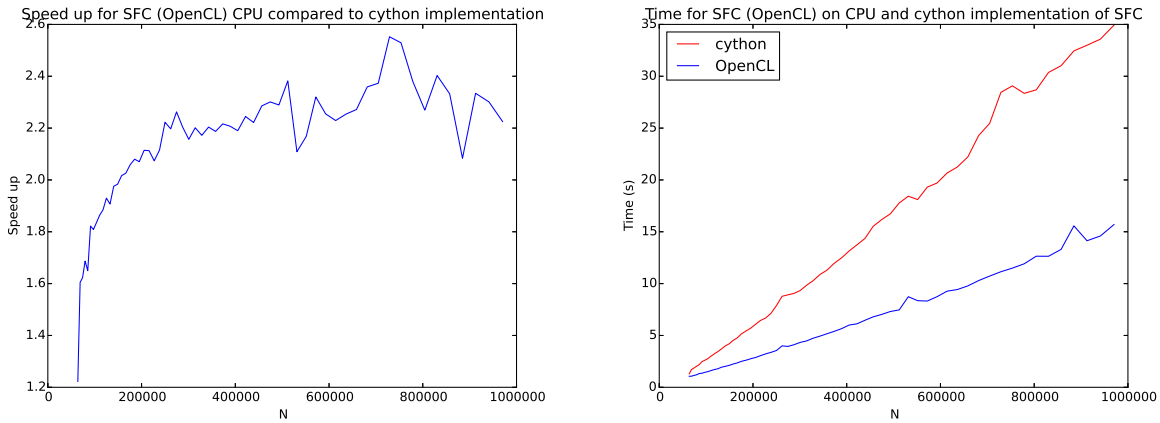


Figure 6.2: Comparison of SFC OpenCL implementation on CPU with Cython implementation



(a) *(b)*

Figure 6.3: Comparison of SFC on GPU with Cython implementation. Shows (a) speed up (b) time taken



(a) *(b)*

Figure 6.4: Comparison of SFC OpenCL implementation on CPU with Cython implementation. Shows (a) speed up (b) time taken

Comparison of SFC on GPU with Cython implementation

Figure 6.3 shows the comparison of SFC's OpenCL implementation run on a GPU with serial Cython implementation. We can see a maximum speed up of around 14x with GPU implementation as compared to Cython serial version.

Comparison of SFC OpenCL implementation on CPU with Cython implementation

Figure 6.4 shows the comparison of SFC's OpenCL implementation run on a CPU with serial Cython implementation. A maximum speed up of around 2.5x with the OpenCL implementation run on CPU with 4 threads as compared to Cython implementation run in serial.

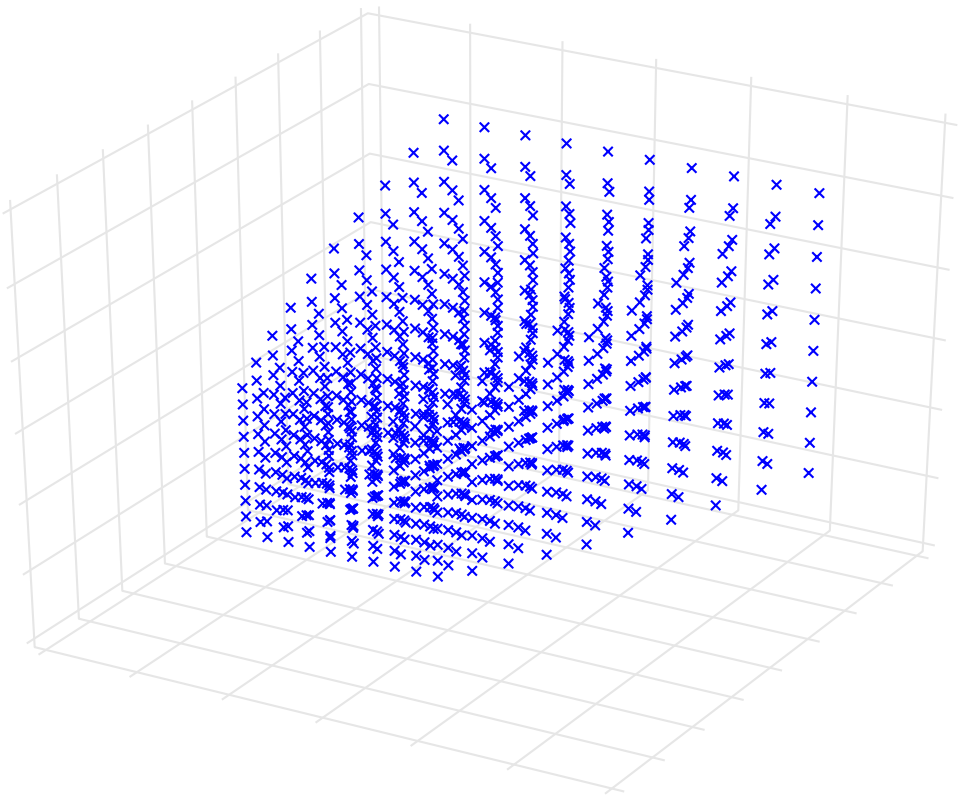


Figure 6.5: Dataset with a gradient of search radius from $h_{\min}$ to $h_{\max}$

6.2.2 Variable search radius

Similar to the previous section, in this section $Nh^3_{\max} = k_{\max}$ and $Nh^3_{\min} = k_{\min}$, such that $k_{\max}$ and $k_{\min}$ remain constant with N.

The dataset in figure 6.5 is such that $h \propto dx$ where dx is the interparticle spacing. This dataset will be called gradient radius dataset.

Comparison of SFC on GPU with Octree

Figure 6.6 shows comparison of SFC's OpenCL implementation run on a GPU with octree run in serial on CPU. Octree is the best performing serial algorithm for this dataset.

Comparison of SFC with Stratified Grids on the GPU

Because we use binary search for implementing stratified grids, we compare it to the SFC implementation using binary search in figure 6.7. Further optimization of stratified grids has been left as a subject for future work.

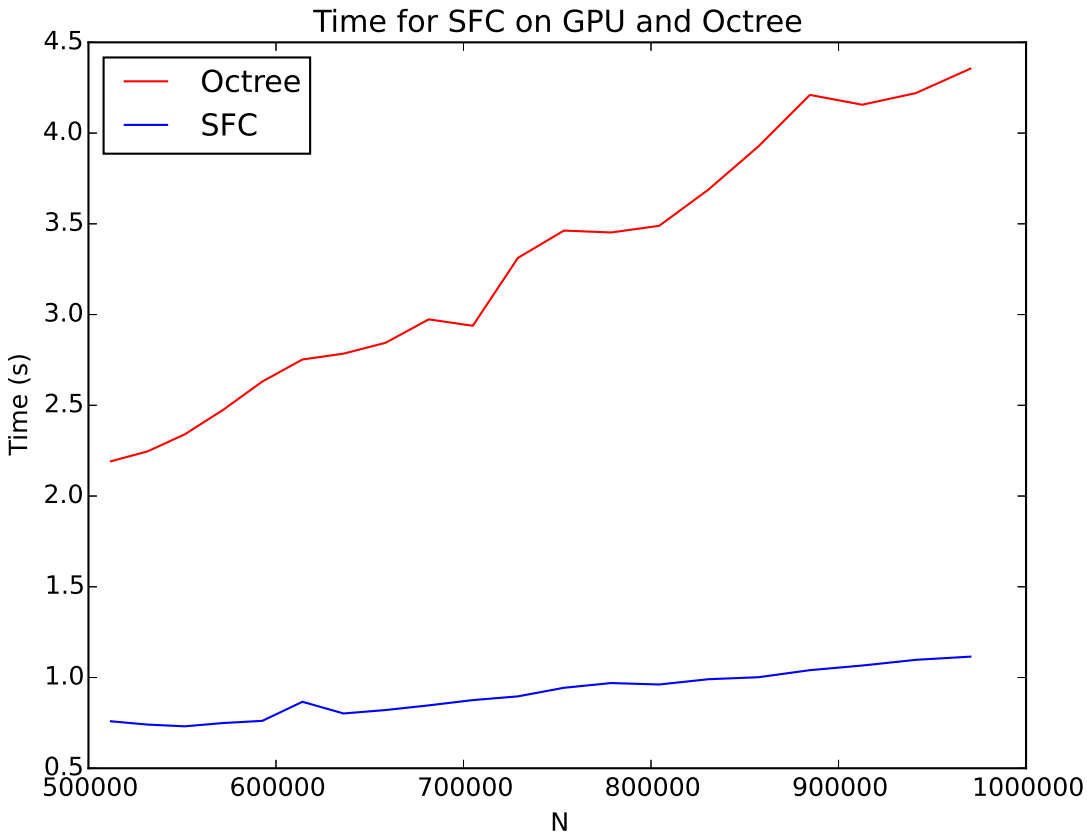


Figure 6.6: Comparison of SFC on GPU with Octree in serial on CPU for $\frac{h_{\max}}{h_{\min}} = 5$

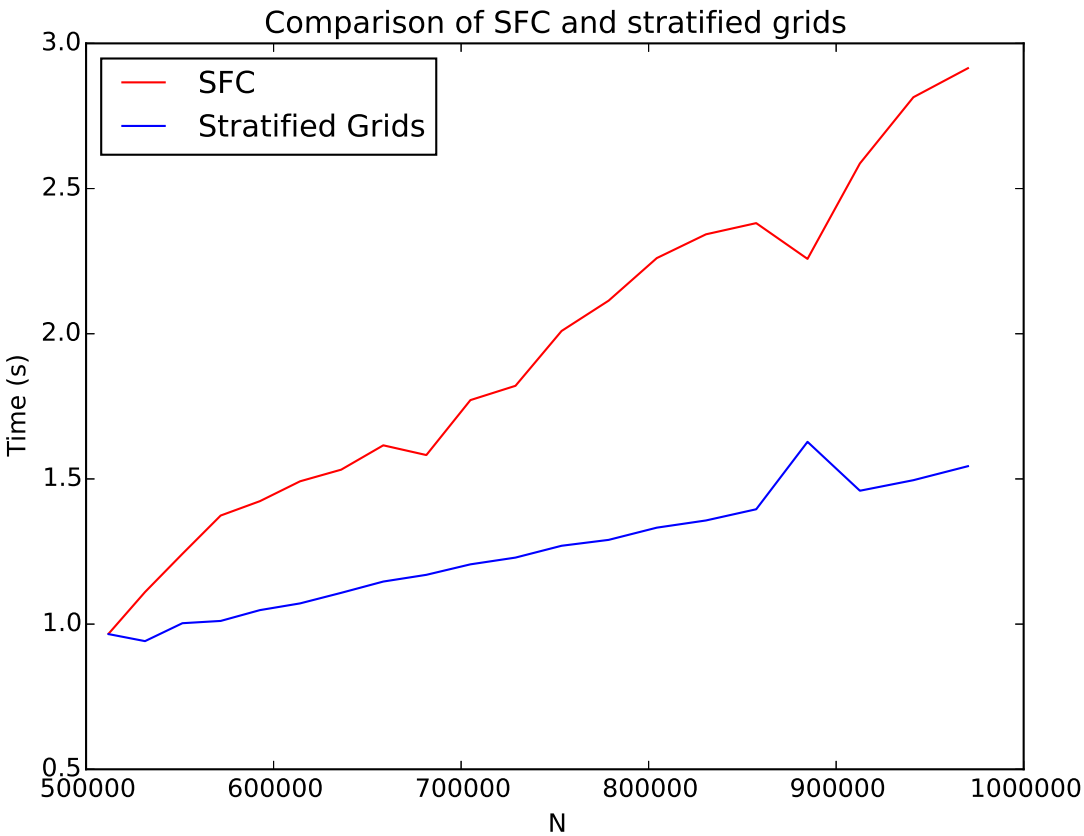


Figure 6.7: Comparison of SFC with Stratified Grids on the GPU for $\frac{h_{\max}}{h_{\min}} = 5$ *and 4 levels in the stratified grids.*

Chapter 7

Summary

The SFC based uniform grid nearest neighbor searching algorithm was implemented in OpenCL. We looked at how memory and speed can be optimized for the best performance on the GPU. The GPU implementation was then compared to the serial implementation using a uniform particle distribution dataset. The OpenCL implementation on CPU was also compared with the serial Cython implementation.

The stratified grids algorithm was implemented on the GPU using the binary search method. Further optimizations in stratified grids is a subject for future work. The stratified grids algorithm on the GPU was compared with SFC using binary search method on the GPU on a gradient radius dataset.

All of the GPU benchmarks were run on an Intel Iris 6200 GPU which is mainly used for graphics processing. A high end GPU used for computing could give much better speed up. For instance Nvidia GTX 1080 is 10x faster [UserBenchmark, n.d.] than Intel Iris 6200 on an N-body gravity simulation benchmark.

Future work involves optimizing the stratified grids implementation and implementing an octree using OpenCL.

Bibliography

Dominguez, J. M., Crespo, A. J. C., Gomez-Gesteira, M., Marongiu, J. C., 2011. Neighbour lists in smoothed particle hydrodynamics. International Journal for Numerical Methods in Fluids 67 (12), 2026–2042.
URL http://dx.doi.org/10.1002/fld.2481

Goswami, P., Schlegel, P., Solenthaler, B., Pajarola, R., 2010. Interactive SPH Simulation and Rendering on the GPU. In: Proceedings of the 2010 ACM SIGGRAPH/Eurographics Symposium on Computer Animation. SCA '10. Eurographics Association, Aire-la-Ville, Switzerland, Switzerland, pp. 55–64.
URL http://dl.acm.org/citation.cfm?id=1921427.1921437

Hernquist, L., Katz, N., 1989. TREESPH - A unification of SPH with the hierarchical tree method. Astrophysical Journal Supplement Series 70, 419–446.

Klöckner, A., Pinto, N., Lee, Y., Catanzaro, B., Ivanov, P., Fasih, A., 2012. PyCUDA and PyOpenCL: A Scripting-Based Approach to GPU Run-Time Code Generation. Parallel Computing 38 (3), 157–174.

Onderik, J., Durikovic, R., 2008. Efficient Neighbor Search for Particle-based Fluids. Journal of the Applied Mathematics, Statistics and Informatics 4 (1), 29–43.

Ramachandran, P., Puri, K., 2013. A framework for parallel particle simulations. In: Proceedings of the 3rd International Conference on Particle-Based Methods.

UserBenchmark, n.d. Speed test your gpu in less than a minute. http://gpu.userbenchmark.com/Compare/Nvidia-GTX-1080-vs-Intel-Iris-Pro-HD-6200-Desktop/3603vsm30277.

Wikimedia, 2008. Lebesgue 3d curve, iteration 2.
URL https://commons.wikimedia.org/wiki/File:Lebesgue-3d-step2.png

Wikipedia, 2017. General-purpose computing on graphics processing units — wikipedia, the free encyclopedia. [Online; accessed 30-April-2017].
URL https://en.wikipedia.org/w/index.php?title=General-purpose_computing_on_graphics_processing_units&oldid=771855802